

Safe Reinforcement Learning-Based Dynamic Obstacle Avoidance for Robotic Manipulators

1st Yangxin Zhao

College of Computer Science
and Technology

Shanghai University of Electric Power

Shanghai, China

1781256551@mail.shiep.edu.cn

2nd Jianliang Mao*

College of Automation Engineering,
Shanghai University of Electric Power
Shanghai, China

*Corresponding author: jl_mao@shiep.edu.cn

3rd Chuanlin Zhang

College of Computer Science
and Technology

Shanghai University of Electric Power

Shanghai, China

clzhang@shiep.edu.cn

Abstract—In many tasks, especially in safety-critical scenarios, ensuring safety is of paramount importance. Simulators offer key advantages by enabling safe exploration, which is essential when training Reinforcement learning (RL) systems in physical environments like human-robot interaction. However, RL algorithms still struggle to succeed beyond simulation, mainly due to insufficient safety guarantees during learning, often leading to failures before the policy reaches its optimal form. To overcome this challenge, we introduce a architecture that integrates temporal modeling with safety constraints. The proposed framework combines: (1) a controller based on model-free RL; (2) a policy network incorporating Long Short-Term Memory (LSTM); and (3) a safety-critical controller designed with Control Barrier Functions (CBF). Our general framework exploits RL to achieve high-performance control without relying on system models, while the LSTM network more effectively captures the temporal dynamics of the environment. In addition, the CBF-based controller not only ensures safety but also constrains policy exploration to enhance learning stability.

Index Terms—Reinforcement learning, dynamic obstacle avoidance, safe RL, control barrier functions

I. INTRODUCTION

Reinforcement learning (RL) focuses on optimizing a policy to achieve maximum long-term rewards [1]. The agent optimizes its policy through repeated interactions with the environment by interacting with the state space, applying actions, and obtaining scalar feedback. Over time, it continuously updates its strategy to enhance long-term performance. This method has proven suitable for solving complex continuous control problems [2].

However, the exploration process in reinforcement learning often involves risks, as the agent may take dangerous actions while exploring the environment, potentially leading to unsafe behaviors [3]. Therefore, Safe RL aims to maximize rewards while ensuring the agent adheres to safety constraints throughout the learning process [4]. This becomes particularly critical in complex or high-risk tasks, where the agent needs to not only learn effective policies in a simulated environment but also ensure that these policies can be safely transferred to physical agents in the real world, avoiding safety issues due to discrepancies between simulation and reality.

This work was supported in part by the National Natural Science Foundation of China under Grant 62203292.

Existing methods in safe reinforcement learning introduce safety through reward shaping, constrained policy optimization and expert guidance [5], but model-free approaches generally lack safety guarantees during training, risking violations in early stages. Model-based strategies using Lyapunov methods can provide formal safety, yet often overlook exploration and performance optimization [6]. Some works ensure safety via backup controllers [7], though this restricts exploration. Recent studies combine model-based safety with model-free learning to improve training safety [8]. However, many existing methods still depend on fallback controllers that lack guidance for policy learning, thereby reducing overall efficiency.

This paper presents an integration of model-free RL algorithms and Control Barrier Functions (CBF), striving to balance safety guarantees with efficient exploration. By incorporating Long-Short-Term Memory (LSTM) networks, the framework effectively addresses the challenges of high-dimensional state space and long-term dependencies in the robotic arm obstacle avoidance problem. Additionally, a reward-optimized model saving mechanism is employed to facilitate the selection of the best policy. The CBF component ensures online safety and helps the RL algorithm effectively explore the policy space, gradually adapting to dynamic systems. This framework not only enhances the arm's learning efficiency in dynamic environments but also guides policy exploration through the CBF controller, ensuring safe application in the real world.

II. PRELIMINARIES

A Markov decision process (MDP) with control-affine deterministic dynamics over an infinite time horizon and discounted rewards [9], described through the tuple $(S, A, f, g, h, c, \mu_0, \beta)$, where: S refers to the state space; A represents the action space; $f : S \rightarrow S$ describes the unactuated system dynamics; $g : S \rightarrow \mathbb{R}^{p,q}$ defines the control-affine actuated dynamics; $h : S \rightarrow S$ accounts for the unknown disturbances in the system.

The evolution of the system over time is governed by

$$s_{t+1} = f(s_t) + g(s_t)a_t \quad (1)$$

where $s_t \in S$ and $a_t \in A$ and g represent the known nominal model of the system dynamic. Additionally, $c : S \times A \rightarrow \mathbb{R}$ represents the reward function, $\mu_0 : S \rightarrow \mathbb{R}$ represents the distribution over initial states, and $\beta \in (0, 1)$ represents the discounting coefficient.

A. Reinforcement Learning

A stochastic policy $\pi(a|s)$ is defined, where π represents a mapping that assigns a probability distribution over actions to each state, i.e. $\pi(a|s) = p(a_t = a | s_t = s)$. The expected discounted reward of the policy, denoted as $J(\pi)$, is given by

$$J(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{\infty} \beta^t c(s_t) \right]. \quad (2)$$

Here, $\tau \sim \pi$ refers to the trajectory $\tau = \{s_t, a_t, \dots, s_{t+n}, a_{t+n}\}$. The value function V_π , action-value function Q_π , and advantage function A_π are defined as follows:

$$\begin{aligned} Q_\pi(s, a) &= \mathbb{E}_\pi \left[\sum_{l=0}^{\infty} \beta^l c(s_{t+l}, a_{t+l}) \right], \\ V_\pi(s) &= \mathbb{E}_\pi \left[\sum_{l=0}^{\infty} \beta^l c(s_{t+l+1}, a_{t+l+1}) \right], \\ A_\pi(s, a) &= Q_\pi(s, a) - V_\pi(s) \end{aligned} \quad (3)$$

where the actions a_i are sampled according to the policy distribution $a_i \sim \pi(a|s_i)$. The notation $\mathbb{E}_\pi[\cdot]$ represents the mathematical expectation.

Proximal Policy Optimization (PPO) stabilizes policy updates and avoids drastic changes by optimizing a surrogate loss with either clipping or a KL divergence penalty, leading to simpler and more efficient policy optimization [10]. The optimal policy update is approximated by solving the optimization problem iteratively:

$$\pi_{i+1} = \arg \max_{\pi} \sum_s \rho_{\pi_i}(s) \sum_a \frac{\pi(a|s)}{\pi_i(a|s)} A_{\pi_i}(s, a) \quad (4)$$

where $\rho_{\pi_i}(s)$ refers to the discounted frequency with which state s is visited under policy π_i .

Traditional PPO uses Multilayer Perceptron (MLP), which struggle to capture temporal dependencies in control tasks [11]. To overcome this, we propose an LSTM-based Actor-Critic architecture that encodes long-term historical information to enhance decision-making. The PPO-LSTM training process is shown in Fig. 1. In the Actor network, we use LSTM to process the state sequence, with its hidden state h_t^π serving as the input for policy decision-making, thereby generating the action probability distribution:

$$\begin{aligned} h_t^\pi, c_t^\pi &= LSTM_\pi(s_t, h_{t-1}^\pi, c_{t-1}^\pi), \\ \pi_\theta(a_t|s_t) &= \pi(h_t^\pi). \end{aligned} \quad (5)$$

Similarly, in the Critic network, we use LSTM to process the state sequence to obtain a more stable value function estimation.

Although PPO-LSTM learns effective controllers on several benchmark problems, like other model-free RL methods, it lacks safety guarantees. As a result, our intention is to integrate the model-free RL controller with a model-based CBF controller, enhancing both exploration efficiency and safety.

B. Control Barrier Functions

Let $\mathcal{X}$ denote a safe set characterized as the set of points where a continuously differentiable function $h : \mathbb{R}^n \rightarrow \mathbb{R}$,

$$\begin{aligned} \mathcal{X} &= \{s \in \mathbb{R}^n : h(s) \geq 0\}, \\ \partial\mathcal{X} &= \{s \in \mathbb{R}^n : h(s) = 0\}, \\ \text{Int}(\mathcal{X}) &= \{s \in \mathbb{R}^n : h(s) > 0\}. \end{aligned} \quad (6)$$

Definition 1. For the set $\mathcal{X} \in \mathbb{R}^n$ as defined in (6), $h : \mathbb{R}^n \rightarrow \mathbb{R}$ is said to be a discrete-time CBF designed for dynamical system (1) if it is continuously differentiable, satisfies $(\partial h / \partial s)(s) \neq 0$ for all $s \in \partial\mathcal{X}$ [12], and there exists a strictly increasing, continuous extended class K function α with $\alpha(0) = 0$, such that for all $s \in \mathcal{X}$, the following condition holds for system (1):

$$\sup_{a_t \in A} \underbrace{\left[\frac{\partial h}{\partial s} f(s) + \frac{\partial h}{\partial s} g(s) a_t \right]}_{\dot{h}(s, a_t)} \geq -\alpha(h(s)). \quad (7)$$

The CBF h ensures the system (1) remains in the safe set $\mathcal{X}$, avoiding unsafe regions [13]. If a valid CBF exists, there is a corresponding deterministic controller $u_{CBF} : S \rightarrow A$ that guarantees forward invariance of $\mathcal{X}$ in system (1). That is, if condition (7) holds for all $s \in \mathcal{X}$, the system stays safe.

To enforce this, the objective is to design a controller u_{CBF} that fulfills the CBF constraint. A common method is to solve an optimization problem that adjusts a potentially unsafe desired input u_{des} as little as possible while ensuring safety. More precisely, the condition specified in (7) can be recast as a quadratic program (QP) formulated as follows:

$$\begin{aligned} u^* &= \underset{u \in U}{\operatorname{argmin}} \|u - u_{des}\|^2 \\ \text{s.t.} \quad & \frac{\partial h}{\partial s} f(s) + \frac{\partial h}{\partial s} g(s) u \geq -\alpha(h(s)). \end{aligned} \quad (8)$$

Considering $\alpha(h(s)) = \gamma h(s)$, the modulation function $h(s)$ in CBF influences the conservativeness or aggressiveness of the system through the scalar parameter γ .

III. REINFORCEMENT LEARNING-ENHANCED CBF FOR ADAPTIVE COMPENSATION

In this section, we introduce a control synthesis framework based on RL-CBF quadratic programming. In this paper, we focus on linear-in-form barrier functions of the form $h = p^T s + q$. Based on this formulation, the CBF condition (7) can be cast as the following QP problem, which is solved efficiently:

$$\begin{aligned} (a_t, \epsilon) &= \underset{a_t, \epsilon}{\operatorname{argmin}} \|a_t\|_2 + l\epsilon \\ \text{s.t.} \quad & p^T f(s_t) + p^T g(s_t) a_t + q \geq \gamma h(s_t) - \epsilon \\ & a_{low}^i \leq a_t^i \leq a_{high}^i \text{ for } i = 1, \dots, N \end{aligned} \quad (9)$$

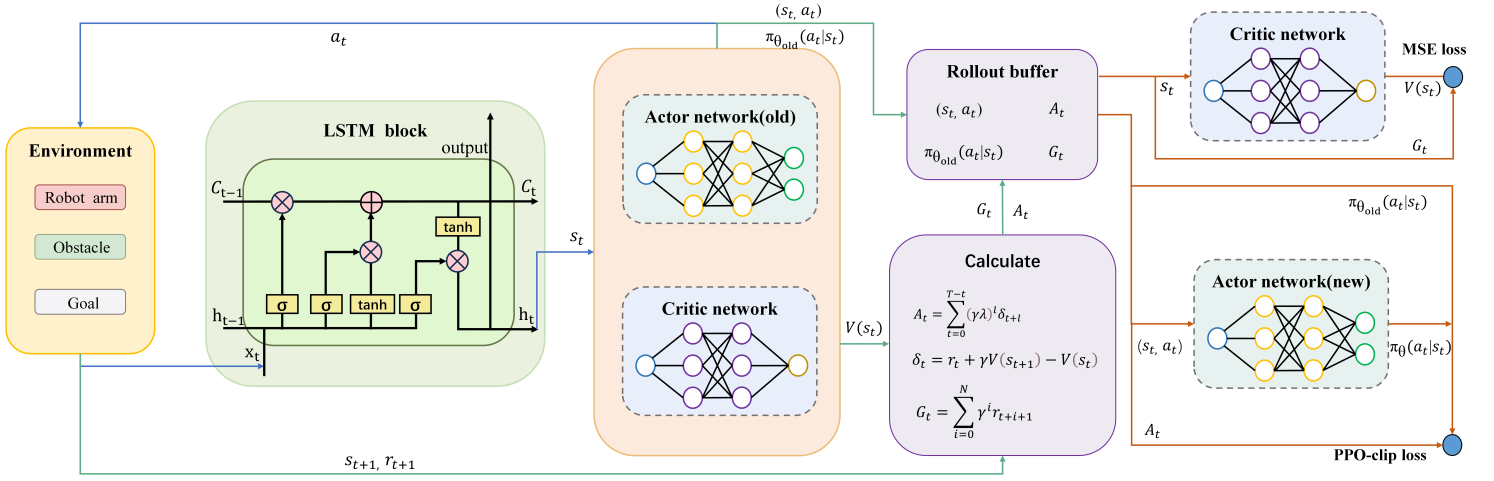


Fig. 1. The PPO-LSTM training process. In the experience collection phase, the Actor network (old) uses an LSTM to process temporal information, generate a policy based on the current state and hidden state, interact with the environment, and store the new hidden state to maintain temporal dependencies. In the update phase, the Actor network (new) recalculates the policy using the LSTM and optimizes it with the PPO-Clip objective function. The Critic network remains unchanged for state value estimation and advantage computation.

Where ϵ represents a slack variable, and $l\epsilon$ is a weighted slack term where ϵ is multiplied by l , $l > 0$ can serve to minimize the safety violations, the last constraint on a_t^i encodes actuator constraints.

To demonstrate our framework, equation (10) defines a suboptimal decision module that fuses a learned policy from model-free RL with a safety-critical control component derived from CBF, as depicted in Fig. 2. This controller ensures a balance between learning-based adaptation and safety enforcement.

$$u_k(s) = u_{\theta_k}^{RL}(s) + u_k^{CBF}(s, u_{\theta_k}^{RL}). \quad (10)$$

Since the reinforcement learning policy $\pi_{\theta_k}^{RL}(a|s)$ is stochastic, the control input $u_{\theta_k}^{RL}(s)$ is a specific action generated by this policy after the k -th iteration, obtained as a sampled instance from its probability distribution. The model-

The CBF controller $u_k^{CBF}(s, u_{\theta_k}^{RL})$, which is influenced by the RL-based control, is formulated as the following QP problem:

$$\begin{aligned} (a_t, \epsilon) = \underset{a_t, \epsilon}{\operatorname{argmin}} & \|a_t\|_2 + l\epsilon \\ \text{s.t.} \quad & p^T f(s_t) + p^T g(s_t) (u_{\theta_k}^{RL}(s_t) + a_t) + q \geq \gamma h(s_t) - \epsilon, \\ & a_{low}^i \leq a_t^i + u_{\theta_k}^{RL(i)}(s_t) \leq a_{high}^i \text{ for } i = 1, \dots, N \end{aligned} \quad (11)$$

Specifically, the result of Lemma 1 in [14] can be extended to (11), where $\epsilon_{max} = \max_{s \in \mathcal{X}} \epsilon$ denotes the maximum barrier violation in (11), i.e., the largest deviation from safety. This implies that the control input $u_k = u_{\theta_k}^{RL} + u_k^{CBF}$ satisfies the barrier condition up to ϵ_{max} , maintaining forward invariance of the set $\mathcal{X}$. If $\epsilon_{max} = 0$, the controller guarantees safety of ϵ_{max} with probability $1 - \delta$. If $\epsilon_{max} > 0$, safety of a relaxed set $\mathcal{X}_\epsilon$ is ensured with the same probability.

IV. SIMULATION VALIDATION

This section presents two experimental setups to validate the proposed approach. The first examines whether the LSTM structure enhances decision-making in dynamic obstacle avoidance. The second integrates CBF into PPO-LSTM to assess its impact on success rate, collision reduction, training efficiency, and final policy performance. All experiments use a UR5 robotic arm in the PyBullet environment.

A. PPO-LSTM

We conduct experiments in PyBullet using a UR5 robotic arm to perform an obstacle avoidance task in a dynamic obstacle environment. The key experimental parameters are as follows: the value of the discount factor is chosen as $\beta = 0.99$, the learning rate is 3×10^{-4} , the PPO update interval is 2048 timesteps, and the batch size is 64. In the PPO-LSTM architecture, the LSTM hidden layer size is set to 64, and the time step length is 10. To verify the efficacy of

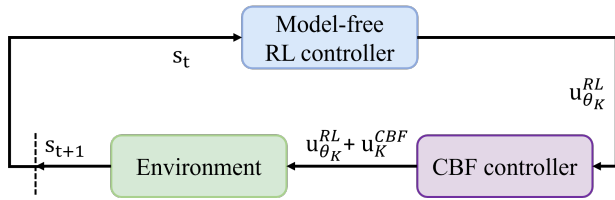


Fig. 2. A control scheme that couples a model-free RL policy with a CBF-based module to ensure safety.

free RL controller $u_{\theta_k}^{RL}(s)$ aims to optimize long-term rewards, but its proposed control actions may not always be safe. To address this, the CBF controller $u_k^{CBF}(s, u_{\theta_k}^{RL})$ adjusts these actions before execution, ensuring system safety with minimal intervention. Essentially, the CBF controller modifies RL policy outputs to ensure safety by constraining actions within the admissible set, preventing potential risks.

the proposed PPO-LSTM method in the robotic arm dynamic obstacle avoidance task, we compared it with the following two methods:

- PPO: Standard PPO with MLP-based Actor and Critic.
- PPO-LSTM (Our method): Incorporates LSTM into the Actor-Critic to capture temporal patterns in obstacle motion.

The experimental results in Fig. 3 show that PPO-LSTM achieves higher average rewards than standard PPO and converges faster in early training, suggesting improved learning of obstacle avoidance behavior. Table 1 further indicates that PPO-LSTM achieves a higher success rate, demonstrating its ability to capture dynamic obstacle motion through temporal modeling, thereby enhancing robustness in complex environments.

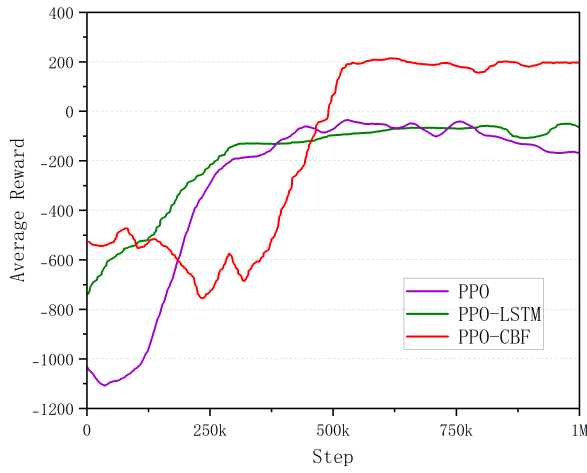


Fig. 3. Comparison of accumulated reward from Dynamic Obstacle Avoidance problem using PPO, PPO-LSTM, PPO-LSTM-CBF.

B. Obstacle Avoidance for a Robotic Manipulator

In this study, we choose the UR5 manipulator as the case for analysis. UR5 is in production and widely used, and its model can be implemented in various simulation and control platforms (e.g., PyBullet), providing a solid experimental foundation for this study.

The forward kinematics function $p = F(q)$ defines the end-effector position of the UR5 manipulator, where $q \in \mathbb{R}^6$ is the joint position vector. The latter will also be considered as the system state. Additionally, we define the system input as the vector of joint velocities, thus $\dot{q} = u$ is adopted for the CBF-based control formulation. The model in (1) still holds, where the drift term $f(q)$ is a zero vector, and the control matrix $g(q)$ is a 6×6 identity matrix.

We can express the safety objective for avoiding a spherical obstacle, centered at p_{obs} with radius R , as a CBF. Considering a parameter R_v , which defines the safety margin around the robot's wrist with radius R_v , the function is defined as follows:

$$h(q) = \|p - p_{obs}\|^2 - (R + R_v)^2, \quad (12)$$

whose rate of change is influenced by the control input in the following manner:

$$\dot{h}(q, u) = 2(p - p_{obs})^T J(q)u. \quad (13)$$

The safety controller can be designed by formulating the following QP:

$$\begin{aligned} \dot{q}^* &= \underset{\dot{q} \in \mathbb{R}^6}{\operatorname{argmin}} \|\dot{q} - \dot{q}_{des}\|^2 \\ \text{s.t. } \dot{h}(q, \dot{q}) &= J_h(q)\dot{q} \geq -\alpha(h(q)). \end{aligned} \quad (14)$$

We define $\alpha(h(q)) = \gamma h(q)$ as in the previous examples with $\gamma \in \mathbb{R}$, and assume the robot joints perfectly track the velocity vector $\dot{q}$. The QP solution guarantees that the UR5 wrist center stays at least a distance of $R_v + R$ away from p_{obs} , regardless of the desired joint velocity $\dot{q}_{des}$.

Fig. 4 illustrate the obstacle avoidance path of the robotic arm. These images show the arm's movement in a dynamic environment, highlighting its ability to successfully navigate around obstacles while reaching the target position. The snapshots capture various stages of the trajectory, demonstrating how the arm adapts to changes in the environment and effectively avoids collisions.

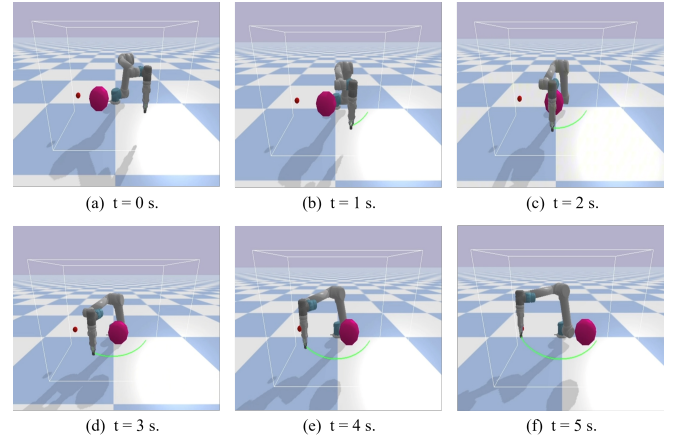
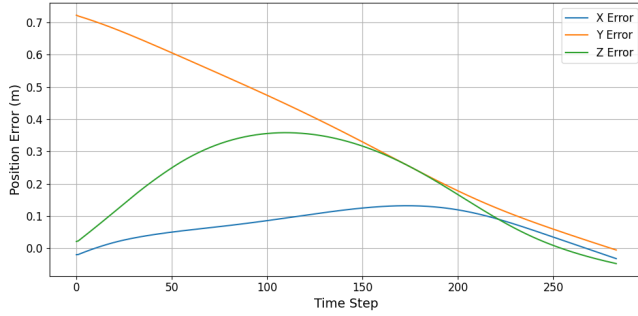


Fig. 4. Simulation snapshots of one episode at different time steps ($t = 1$ to 6), illustrating the motion of the robotic manipulator during the task execution.

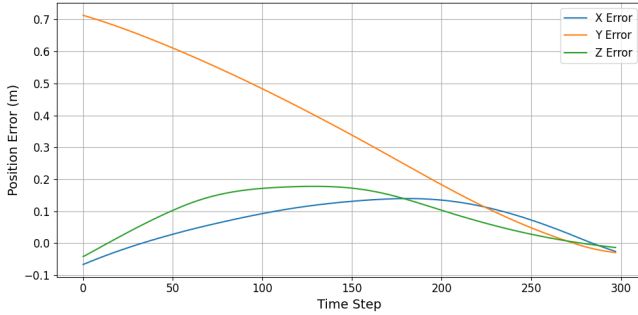
As shown in Fig. 5, the separation between the end-effector of the robot manipulator and its target gradually diminishes. Since the standard PPO algorithm only optimizes rewards, it may execute some unsafe but faster actions, resulting in a shorter task completion time. In contrast, CBF constrains the speed for safe obstacle avoidance, preventing the robotic arm from accelerating too quickly, which may require more time to reach the target.

Table 1 shows the success rate, average reward, minimum distance between the robot end-effector and the obstacle, and computation time during the 100-episode test after training after 500 training episodes under different strategies. It is evident that the method incorporating CBF achieves significantly higher success rate and average reward compared to the others. Although it introduces a slight increase in computation time,

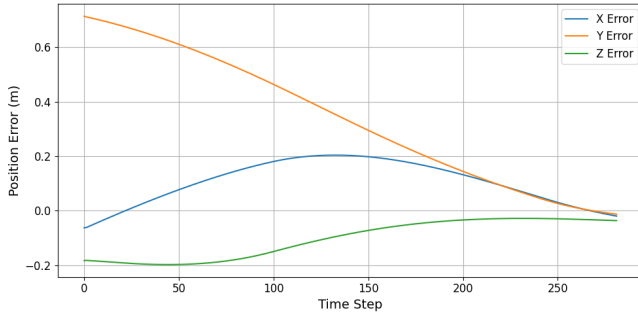
the overhead is minimal and does not noticeably affect the overall task completion. Moreover, the method maintains a larger minimum distance from the obstacle, indicating stronger obstacle avoidance capability and improved safety margin under trajectory prediction and constraint guidance.



(a) PPO



(b) PPO-LSTM



(c) PPO-LSTM-CBF

Fig. 5. Real-time tracking errors of the end-effector in a collision-free reaching task for three different approaches.

TABLE I
COMPARISON UNDER COLLISION-FREE REACHING TASK

Policy	SR(%)	Reward	Avg compute time	distance
PPO	0.68	-166.4916	4.88471	0.05
PPO-LSTM	0.88	-63.71836	5.28316	0.07
PPO-LSTM-CBF	0.93	196.93913	5.42946	0.11

V. CONCLUSION

In this paper, we have proposed a safe RL framework and developed a controller synthesis algorithm that has guaran-

teed safety and improved exploration. This framework has combined the exploration capability of model-free RL, the temporal information processing of LSTM, and the safety constraints of CBF, enabling the robotic arm to effectively avoid unsafe states during learning and to optimize obstacle avoidance behavior. A key assumption has been the existence of an effective safe set $\mathcal{X}$ that remains forward invariant. If a valid safe set has not been available, unsafe states may have arisen, although the controller has been able to take appropriate actions to mitigate risks. Future work will focus on not only learning the controller but also learning the safety set.

REFERENCES

- [1] Y. Vasilenko and P. Kormushev, "Reinforcement learning for robotic manipulation: A review," *Robotics and Autonomous Systems*, vol. 150, p. 103996, 2022.
- [2] H. Zhang, C. Zhao, and J. Ding, "Robust safe reinforcement learning control of unknown continuous-time nonlinear systems with state constraints and disturbances," *Journal of Process Control*, vol. 128, p. 103028, 2023.
- [3] H. Xu, X. Chen, H. Chen, Q. Sun, and Z. Han, "Safe exploration in reinforcement learning: A survey," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 9, pp. 4040–4056, 2022.
- [4] M. Li, Y. Zhang, Y. Li, M. Zhu, and H. He, "Safe reinforcement learning: A survey," *IEEE Access*, vol. 9, pp. 38 376–38 390, 2021.
- [5] P. Mertikopoulos, K. Georgiou, and N. Mavridis, "Safe reinforcement learning via risk-sensitive policy optimization," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 3, pp. 937–951, 2022.
- [6] Q. Liu, J. Mao, L. Han, C. Zhang, and J. Yang, "Predictive observer-based dual-rate prescribed performance control for visual servoing of robot manipulators with view constraints," *IEEE Transactions on Cybernetics*, vol. 55, no. 5, pp. 2424–2436, 2025.
- [7] L. Sun, P. Zhang, H. Liu, and Z. Chen, "Safe model-based reinforcement learning with lyapunov stability and backup controllers," *IEEE Transactions on Robotics*, vol. 38, no. 4, pp. 1132–1148, 2022.
- [8] S. Pintea, M. Faggiani, R. Li, A. Barrau, A. Agha-mohammadi, and R. D'Andrea, "Safe exploration in model-based reinforcement learning with uncertainty-aware risk constraints," *IEEE Robotics and Automation Letters*, vol. 6, no. 4, pp. 7874–7881, 2021.
- [9] J. Chen, A. Tamar, Z. Pan, A. Miele, and A. Agarwal, "Model-based reinforcement learning with control-affine systems," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 30, no. 4, pp. 1135–1147, 2019.
- [10] A. Raffin, A. Hill, A. Gleave, C. Chan, R. Zhokh, and P. Abbeel, "Stable baselines3: Reliable reinforcement learning implementations," *Journal of Machine Learning Research*, vol. 22, no. 107, pp. 1–8, 2021.
- [11] S. Agarwal, M. A. Rodriguez, and R. Buyya, "A deep recurrent-reinforcement learning method for intelligent autoscaling of serverless functions," *IEEE Transactions on Services Computing*, vol. 17, no. 5, pp. 1899–1910, 2024.
- [12] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," *IEEE Transactions on Automatic Control*, vol. 62, no. 8, pp. 3861–3876, 2016.
- [13] F. Ferraguti, C. T. Landi, A. Singletary, H.-C. Lin, A. Ames, C. Secchi, and M. Bonfe, "Safety and efficiency in robotics: The control barrier functions approach," *IEEE Robotics and Automation Magazine*, vol. 29, no. 3, pp. 139–151, 2022.
- [14] R. Cheng, G. Orosz, R. M. Murray, and J. W. Burdick, "End-to-end safe reinforcement learning through barrier functions for safety-critical continuous control tasks," *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 3387–3395, 2019.